Introduction to the AMD Vivado™ Logic Analyzer

2023.2

AMD
together we advance_

1

Objectives

After completing this module, you will be able to:

OBJECTIVE 01

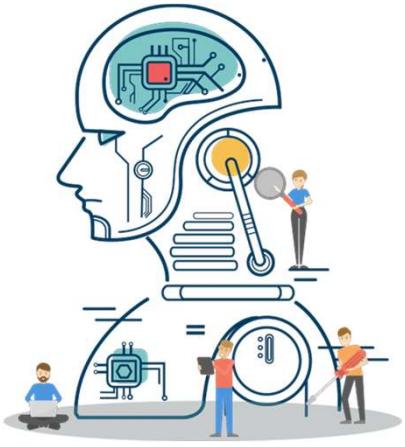
Describe the AMD Vivado™ logic analyzer (VLA) and its fundamental components

OBJECTIVE 02

Enumerate the benefits of the Vivado logic analyzer

OBJECTIVE 03

Describe the basic probing flows to debug a design



AMD
together we advance_

2

Logic Analyzer Basics

Designers spends more than 40% of their time in debugging

Vivado™ logic analyzer debug solution reduces the time required for verification and debugging

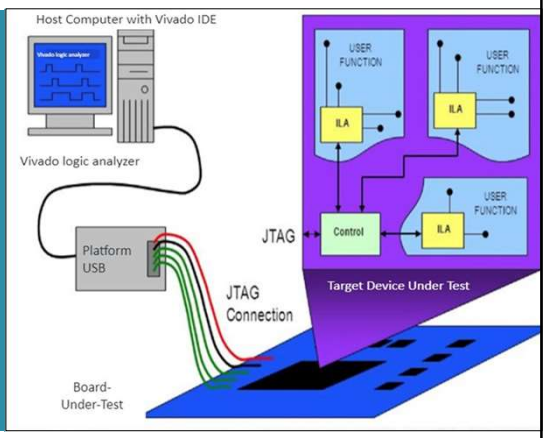
3

Logic Analyzer Basics

Designers can maximize productivity with a solid understanding of the underlying function of this Vivado™ debug tool

Accelerate your success of design by:

- Reducing risk of issues
- Rapidly locating problems
- Fixing the bug quickly



4

Motivation for the Vivado Logic Analyzer Tool

01 Functionality of a conventional logic analyzer can be built into any AMD FPGA

02 The triggering and data collection capability of a logic analyzer can be implemented using the Vivado™ debug cores

03 Vivado debug cores do not consume any I/O pins

04 VLA allows different triggering scenarios

05 Single JTAG connection to the PC can be used for programming

5

Vivado Logic Analyzer

Vivado logic analyzer is integrated into the Vivado Design Suite

6

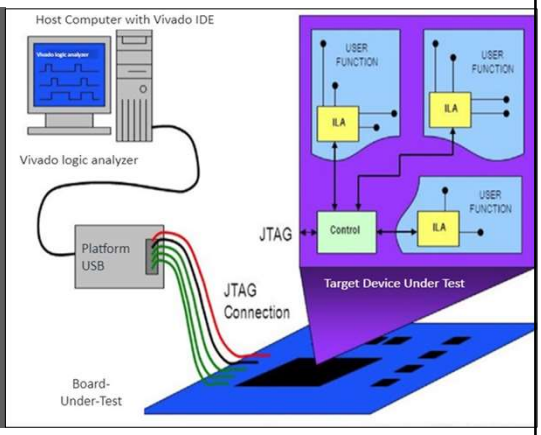
Vivado Logic Analyzer

Powerful yet easy-to-use debug tool

Synergistic role with simulation

Vivado™ logic analyzer can be used in three contexts:
- Verification
- Debugging
- Data capture

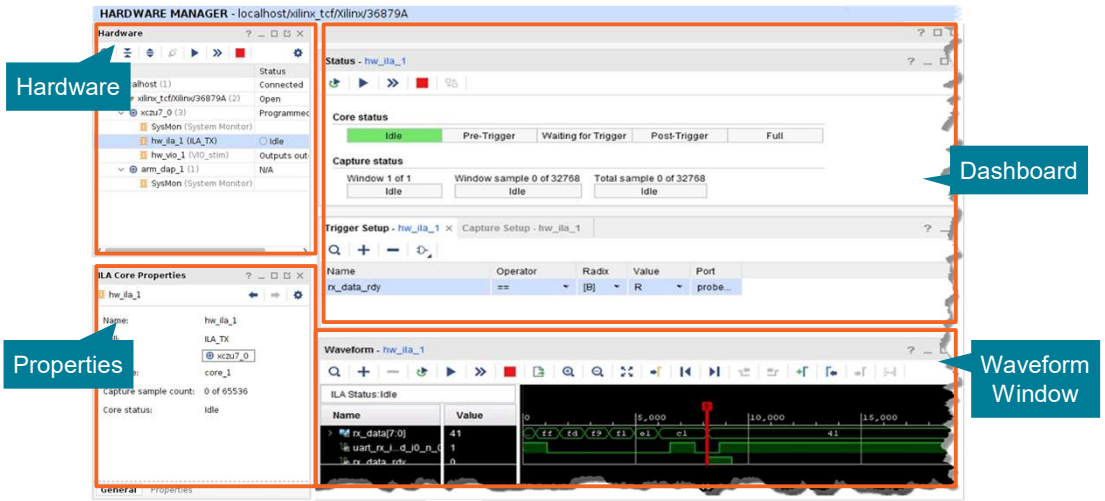
Interact with debug IP cores



AMD together we advance_

7

Vivado Logic Analyzer Tool



Vivado™ Logic Analyzer Tool

AMD together we advance_

8

VLA Comprised of…

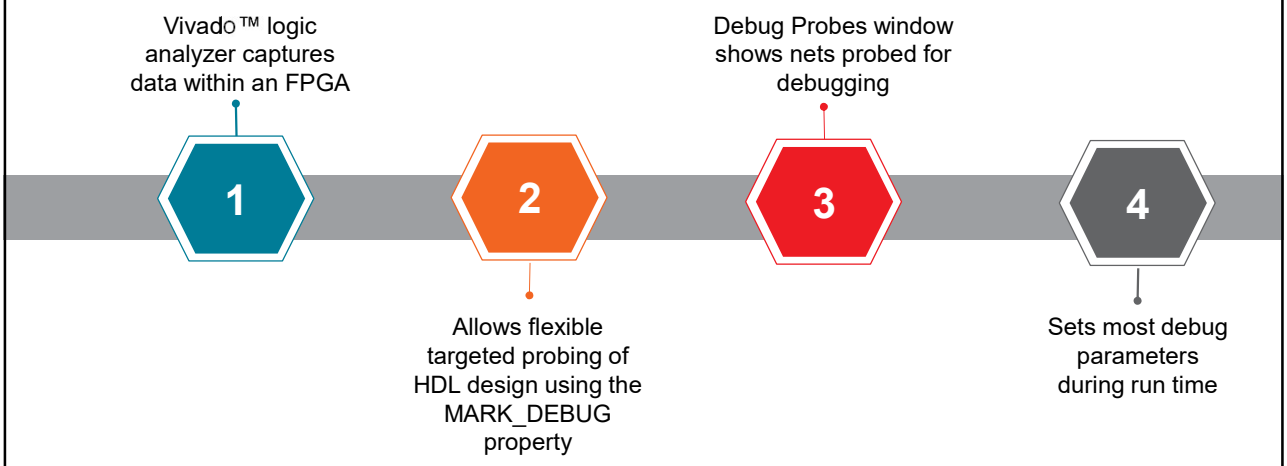
Comprised of two basic items:

- Debug cores
  - ILA, System ILA, VIO, JTAG to AXI Master, and IBERT, etc.
- Debug flows
  - HDL instantiation and netlist insertion flows

AMD
together we advance_

9

Benefits of Logic Debug



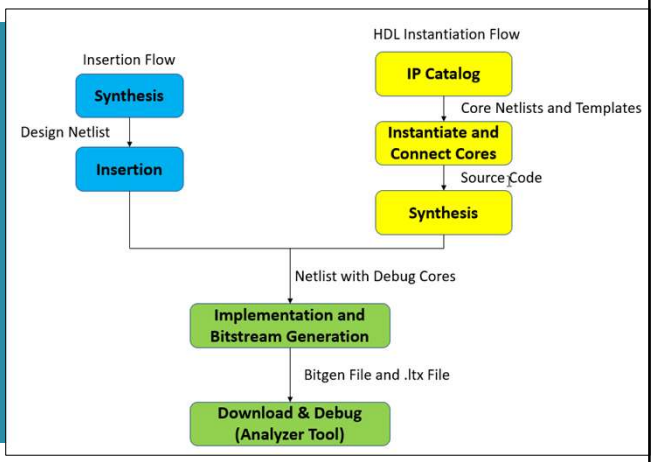
AMD
together we advance_

10

Vivado Logic Analyzer Probing Flows

Two Vivado™ logic analyzer probing flows:
- HDL instantiation flow
- Netlist insertion flow

HDL instantiation flow and netlist insertion flows can be mixed



AMD together we advance_

11

HDL Instantiation or Insertion Flow

Debug flows: a comparison

HDL Instantiation Flow	Netlist Insertion Flow
Allows user to connect to any signal User must modify HDL code to instantiate cores	Recommended flow: easy to add/remove debug core Vivado™ tool auto-inserts debug core into post-synthesis netlist

AMD together we advance_

12

Apply Your Knowledge

Q The Vivado™ logic analyzer can _____. (Select all that apply)

- ☐ Implement triggering and data collection capabilities
- ☐ Is used only for simulation
- ☐ Interact with debug IP cores and help with debugging the design
- ☐ Help with capturing real-time data from the design

13

Apply Your Knowledge

Q Why is the netlist insertion flow recommended as the easier and more flexible option for adding logic analyzer cores in the Vivado™ IDE?

- ○ It allows probing at different design levels and requires no resynthesis
- ○ It provides a fully automatic core insertion process
- ○ It is limited to probing at the HDL design level only
- ○ It requires modification of HDL code for core insertion

14

Summary

01 The Vivado™ logic analyzer is extremely effective in the verification and debugging of FPGA designs

02 Debug cores can implement triggering and data collection capabilities of the logic analyzer

03 The logic analyzer is comprised of two main items:
- Debug cores
- Flows for generating and inserting debug cores

04 Insertion and HDL instantiation flows are the Vivado IDE debug probing flows

AMD
together we advance_

15

Introduction to Triggering

2023.2

AMD
together we advance_

16

Objectives

After completing this module, you will be able to:

OBJECTIVE 01

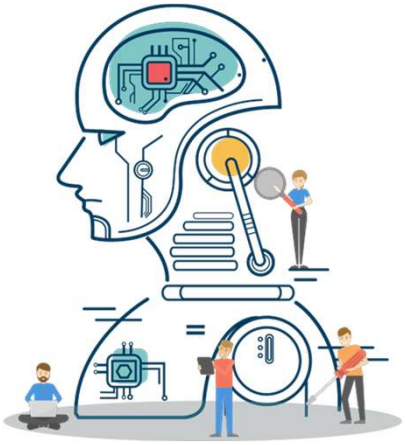
Describe the trigger mechanism of the AMD Vivado™ logic analyzer

OBJECTIVE 02

Explain how to use the Run Trigger option

OBJECTIVE 03

Display captured data using the waveform view

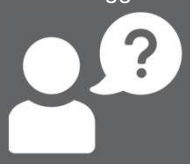


17

Triggering

Triggering is…

- Capturing data when a condition is met
- Condition to set the trigger can be anything
- Data can be displayed either prior to or after the occurrence of the trigger or both



18

Triggering

1. Plays a key role in debugging
2. Enables the ILA cores to capture data at the proper time
3. Helps in finding a specific event and avoids the uninteresting data
4. Two or more different ILAs can be used

19

Setting Up Triggering

Once settings are selected, hardware is generated and triggering is fixed

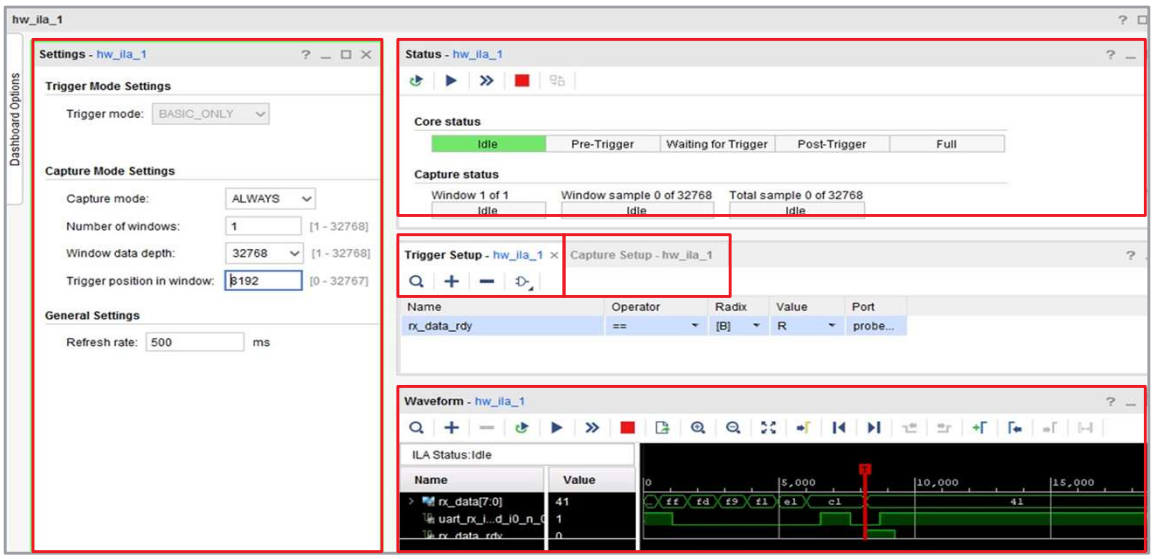
Tailor trigger to specific need by combining core capabilities

Configuring the ILA cores:
- HDL instantiation flow
- Netlist insertion flow

Analyzer tool with the Debug Probes window

20

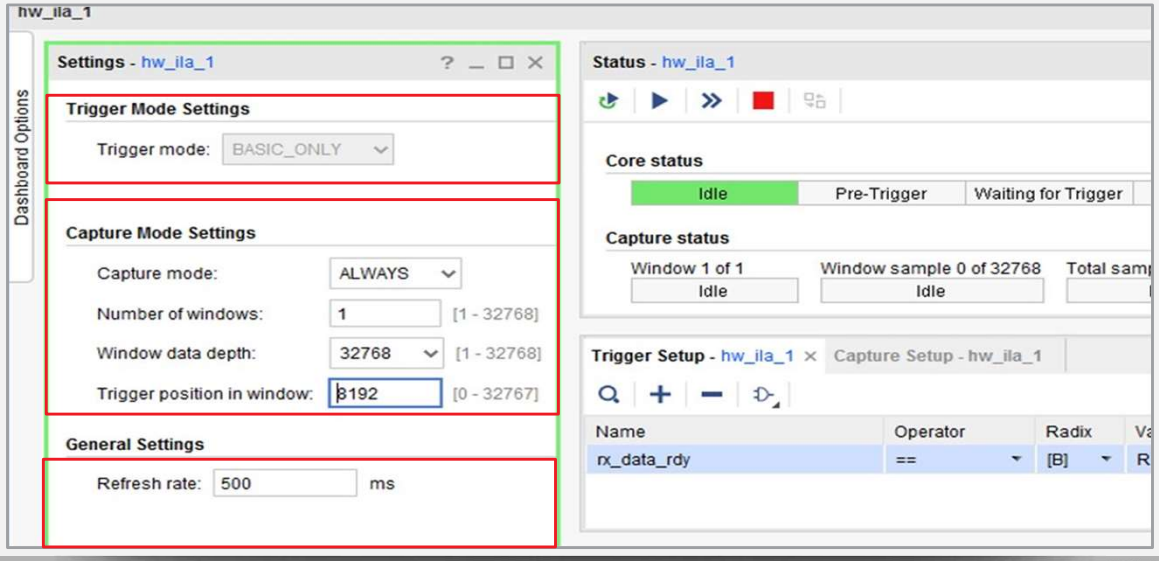
ILA Dashboard



AMD
together we advance_

21

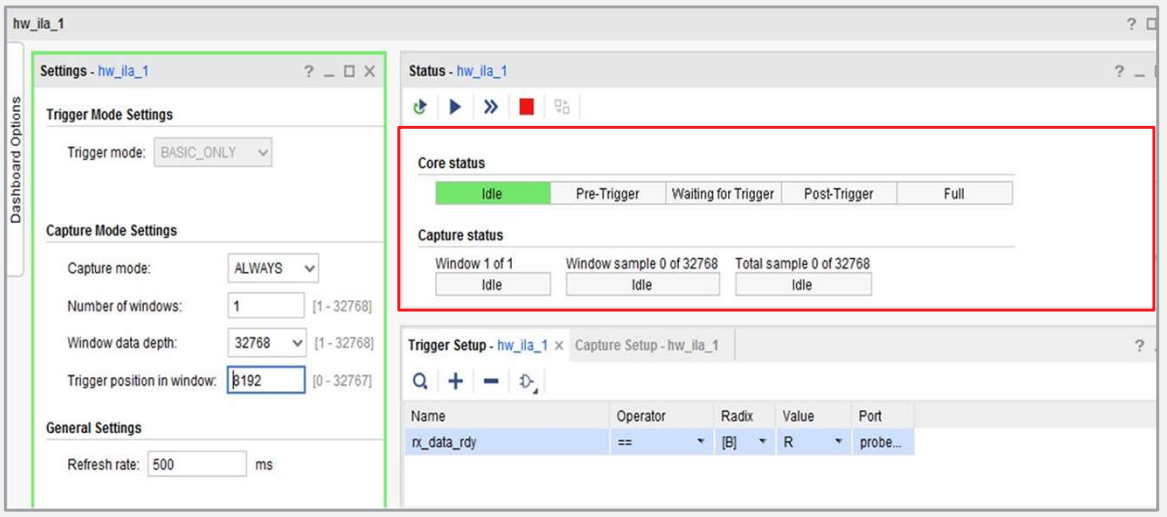
ILA Dashboard – Settings



AMD
together we advance_

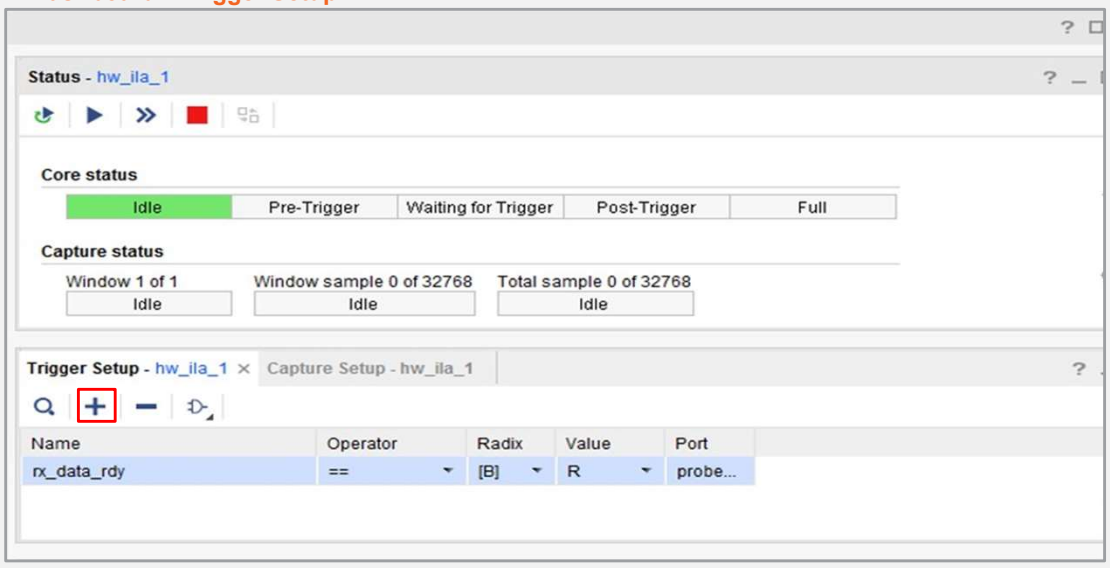
22

ILA Dashboard – Status



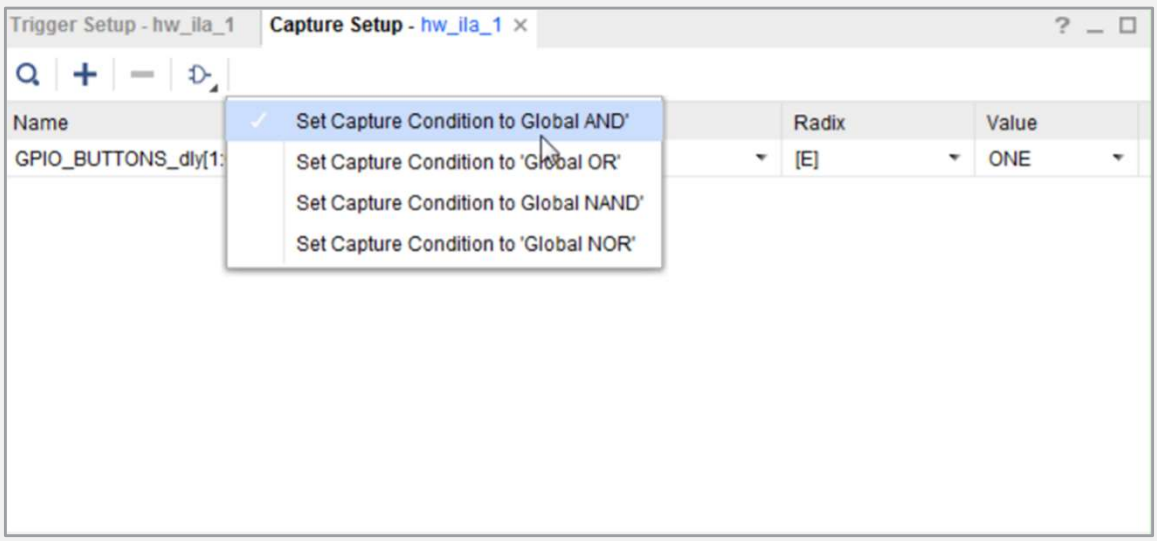
23

ILA Dashboard – Trigger Setup



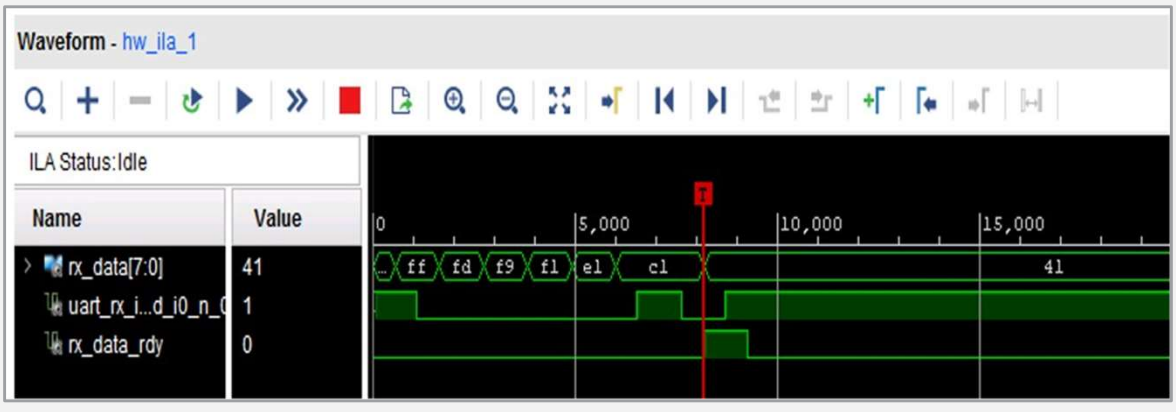
24

ILA Dashboard – Capture Setup



25

ILA Dashboard – Waveform

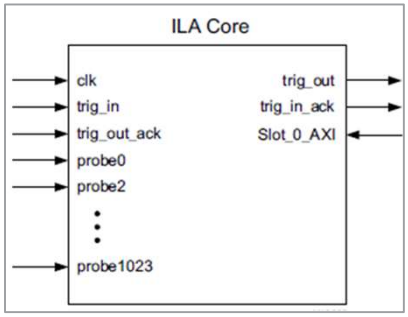


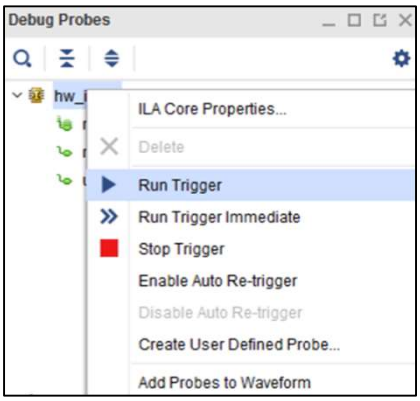
26

ILA Terminology

- 1024 unique probes
- Each of width ranging from 1 to 4096
- ILA specifies the match type and sensitivity

Once the trigger condition is met, the ILA core displays the waveform





27

Displaying Captured Data

ILA waveform viewer provides a powerful way to analyze data

After successfully triggering, the Vivado™ IDE automatically populates a corresponding waveform viewer

28

Apply Your Knowledge

Q Why is triggering important in data capture and analysis?

- ○ It helps maintain a continuous data flow
- ○ It ensures data is always displayed after the trigger
- ○ It allows data capture to be synchronized with specific events or conditions
- ○ It has no impact on data analysis

AMD together we advance_

29

Apply Your Knowledge

Q When a run trigger is issued to the ILA core, it _____.

- ○ Arms the trigger
- ○ Waits for the trigger
- ○ Displays in the waveform window
- ○ All of the above

AMD together we advance_

30

Summary

01 The ILA core can trigger on many events with flexible event combinations

02 The Vivado™ logic analyzer leverages the ILA core's sampling capability

03 Triggering
- Enables tailoring for specific sensitivity
- Supports the combining of triggering conditions

04 Data can be displayed as a waveform
- Each ILA needs a separate waveform window

AMD
together we advance_

31

Debug Cores

2023.2

AMD
together we advance_

32

Objectives

After completing this module, you will be able to:

OBJECTIVE 01

Describe the ILA core and its properties

OBJECTIVE 02

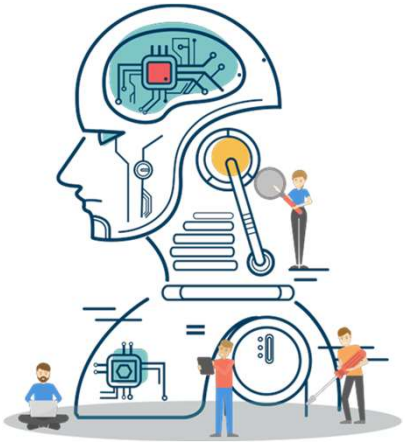
Describe VIO and IBERT core usage

OBJECTIVE 03

Describe what the debug core hub is and why it is used

OBJECTIVE 04

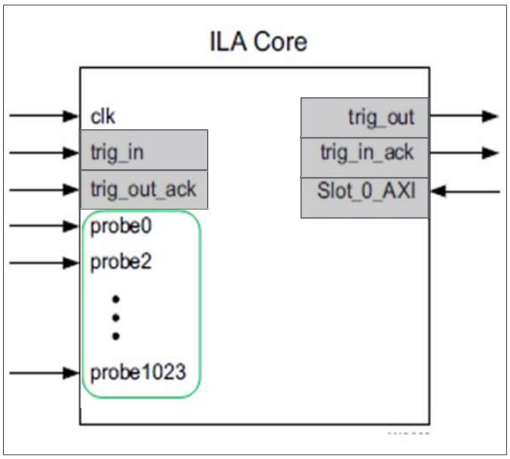
Describe debug core hub insertion and customization



33

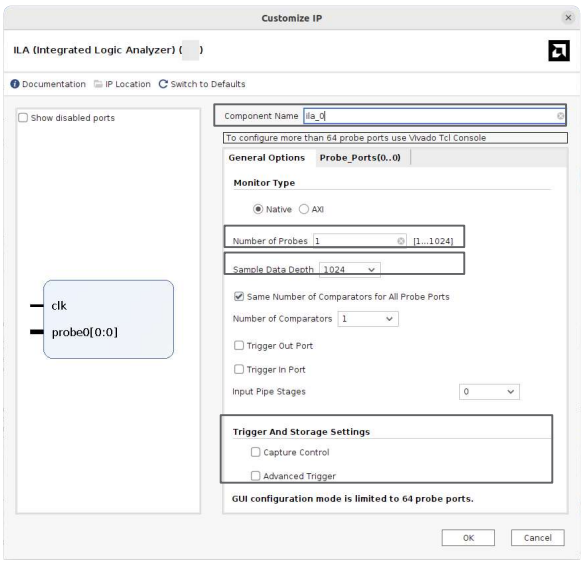
Integrated Logic Analyzer (ILA)

- Used to monitor any design
- Signals are connected to ILA core clock and probe inputs
- Signals sampled at design speed and stored in block RAM
- User configurable up to 1024 probe ports
- Up to 4096 signals per probe port
- Monitor types: Native and AXI
- Communication with the ILA core is conducted using an auto-instantiated debug core hub



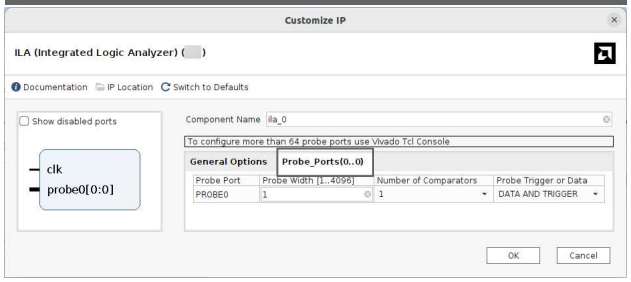
34

ILA Core Wizard



User-selectable parameters are:
- Component name
- Number of probe ports
- Sample data depth
- Trigger and storage settings
- Width for each probe input

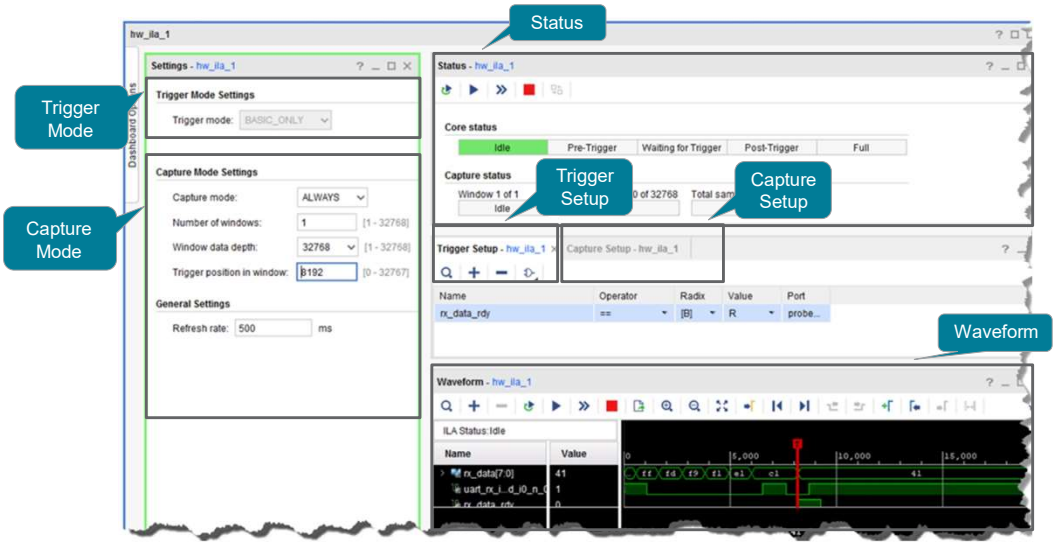
ILA debug core supports:
- HDL instantiation flow
- Netlist insertion flow



AMD together we advance_

35

ILA Dashboard

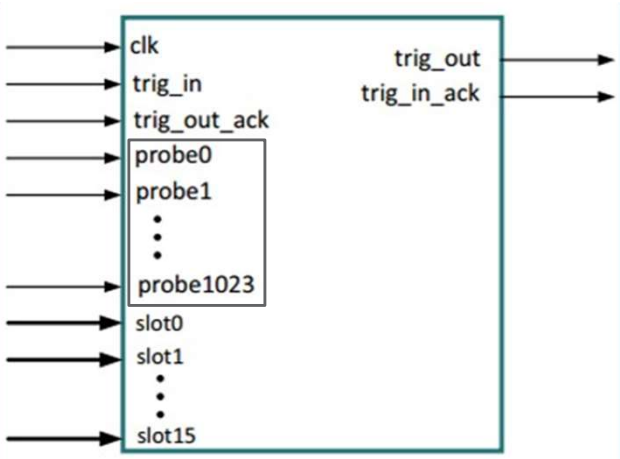


AMD together we advance_

36

System ILA Core

- Used to monitor the internal signals and interfaces
- Allows in-system debugging
- Signals and interfaces in the FPGA design are connected to the System ILA probe and slot inputs
- Sampled at design speeds and stored using on-chip block RAM
- User configurable up to 1024 probe ports
- Up to 4096 signals per probe port



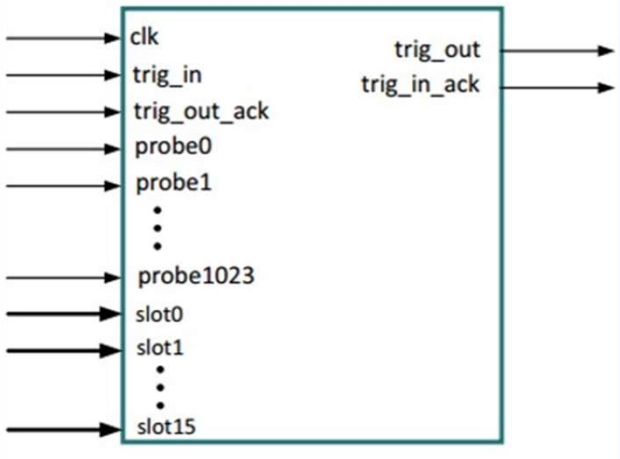
System ILA Core

37

System ILA Core

Core parameters specify:

- Number of probes and interface slots
- Interface types, trace sample depth
- Data and trigger properties of probes and interfaces
- Number of comparators and the width for each probe



System ILA Core

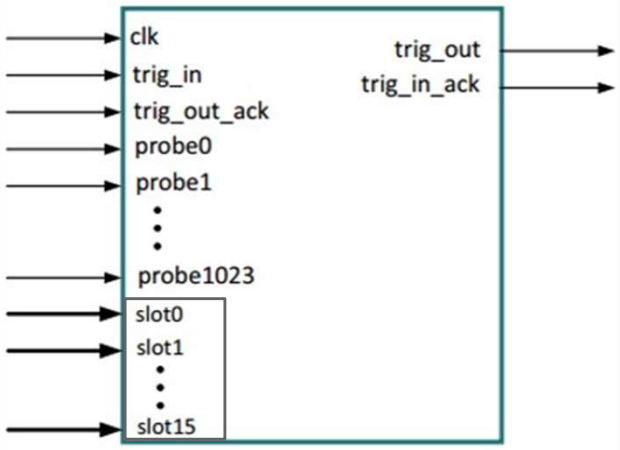
38

System ILA Core

Monitor types:
- Native
- Interface

Slot_0_AXI → AXI interface

Up to 16 interface slots

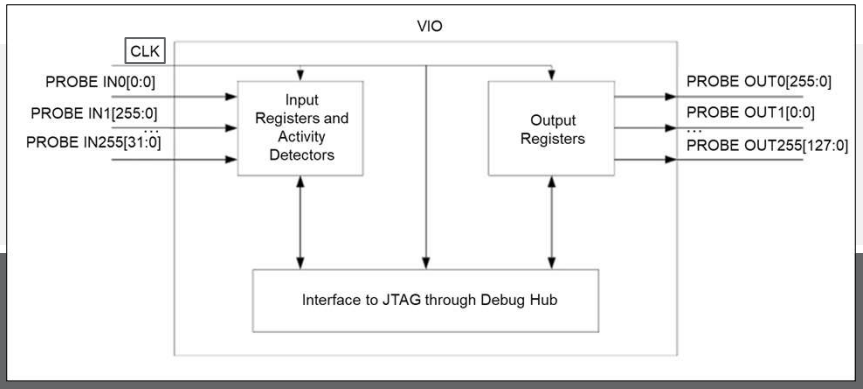


System ILA Core

AMD together we advance_

39

Virtual Input/Output (VIO)



Customizable core

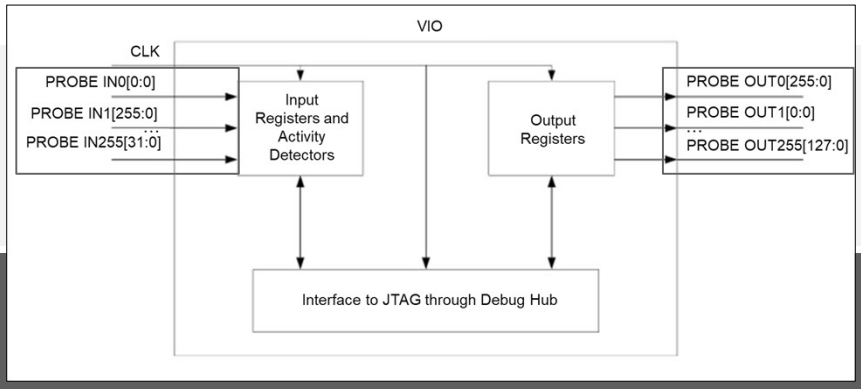
No on-chip or off-chip RAM is required

Presents and captures data on the design clock

AMD together we advance_

40

Virtual Input/Output (VIO)



Insert “virtual” pins into the design

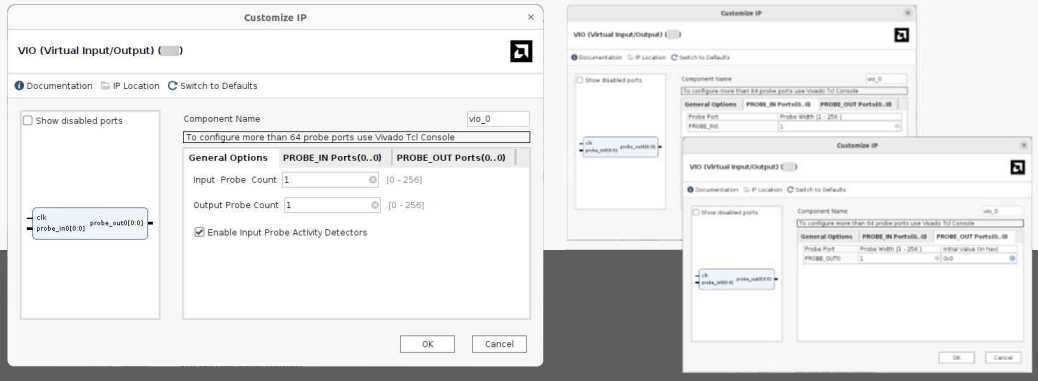
Can produce up to 65 input and output probes

Each port can be 256 bits wide

Inputs and outputs can be visualized in different ways

Cores can be used as virtual LEDs, pushbutton, or toggle switches

VIO Core Wizard



User-selectable parameters:

- Component name
- Number of input and output probe ports
- Width of each probe port

This debug core supports only the HDL instantiation flow

Virtual Input/Output (VIO)

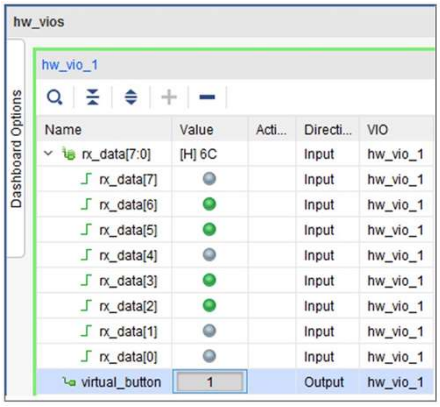
VIO Ports and Parameters | **VIO Dashboard**

Signal Name	I/O	Description
CLK	I	Design clock used to register input ports and output ports. This is required
PROBE_IN<m> [<n> - 1:0]	I	Input probe port number <m> (where <m> can be 0 to 256) of width <n> (where <n> can be 1 to 256). For a 1-bit wide port, use PROBE_IN<m>[0:0]
PROBE_OUT<m>[<n> - 1:0]	O	Output probe port number <m> (where <m> can be 0 to 256) of width <n> (where <n> can be 1 to 256). For a 1-bit wide port, use PROBE_OUT<m>[0:0]

Virtual Input/Output (VIO)

VIO Ports and Parameters | **VIO Dashboard**

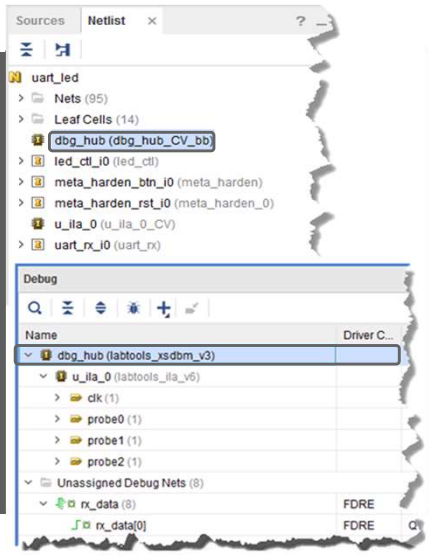
- Shows all status and control information pertaining to a given VIO core
- Drag-and-drop VIO probes from the Debug Probes window
- Monitor “virtual” pins in your design
- Inputs appear as LEDs or text
- Outputs appear as toggle buttons



Debug Core Hub

- Interfaces between JTAG BSCAN and Vivado™ Design Suite debug cores
- Only one dbg_core_hub is required in a design
- Debug cores are connected to provide connectivity
- Designer can view and change BSCAN user scan chain
- Designer must ensure that the clock going to the dbg_hub is a free-running clock
- `connect_debug_port` Tcl command is used to connect the clk pin

Debug Core Hub Insertion | **Debug Core Hub Customization**

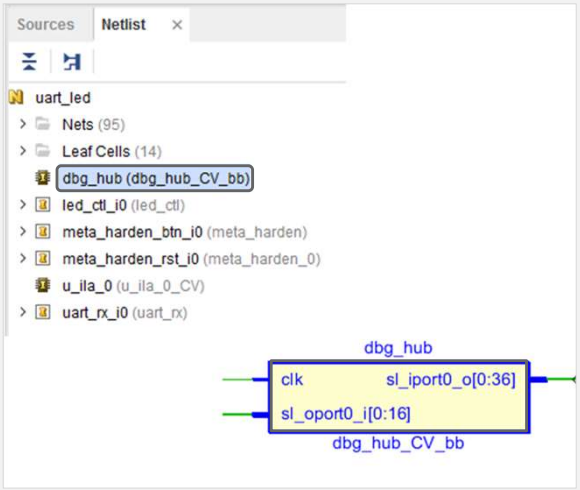


AMD
together we advance_

45

Debug Core Hub Insertion

- Debug cores are inserted into the netlist initially as black boxes (not implemented yet)
- Instantiated at the top level of the design hierarchy
- Netlist and Schematic views display new cores
- Black-box module **dbg_hub** displayed in Netlist view

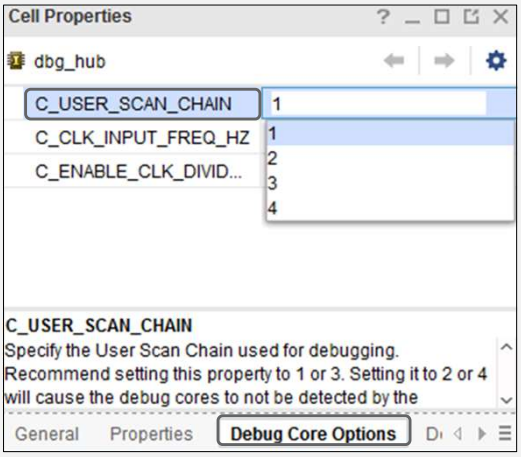


AMD
together we advance_

46

Debug Core Hub Customization

- To view or change the BSCAN user chain index of dbg_hub: Select **dbg_hub** to open the Cell Properties window and click **Debug Core Options**
- View or change the **C_USER_SCAN_CHAIN** property



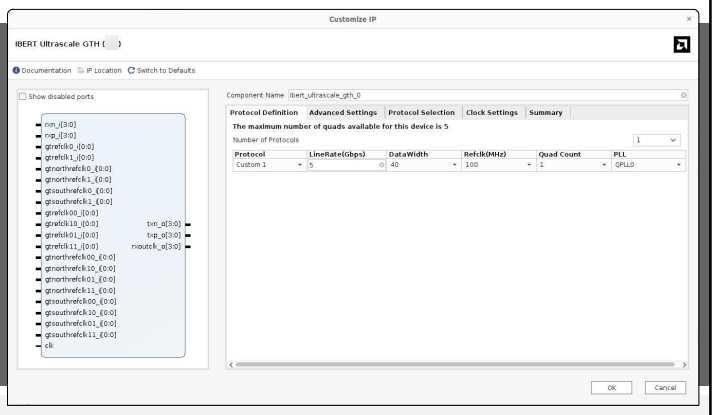
Debug Bridge

The Debug Bridge IP core is…

- Provides multiple communication options with debug cores
- Enables remote debugging over a Xilinx Virtual Cable (XVC) server
- Enables tandem debug with field update designs

Integrated Bit Error Ratio Tester (IBERT)

- Debugs, verifies, and optimizes high-speed serial communication channels
- Provides access to the multi-gigabit transceivers
- Performs bit-error ratio analysis on channels composed of MGT/GTP/GTX/GTH components
- Analyzer tool provides user-adjustable parameters to test various configurations



Recommendations

AMD
together we advance_

49

Recommendations

- Use the IBERT Serial Analyzer design to address a range of in-system debug and validation problems—from simple clocking and connectivity issues to complex margin analysis and channel optimization issues
- Use the IBERT Serial Analyzer to measure signal quality after receiver equalization

AMD
together we advance_

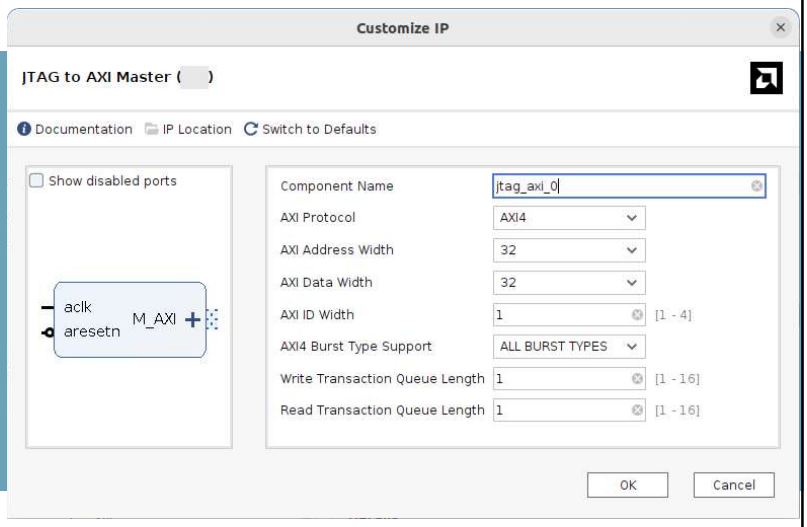
50

JTAG to AXI Master

Used to generate AXI transactions

Support for 32- and 64-bit wide AXI and AXI-Lite interfaces

Recommendation: Use this core to generate AXI transactions and debug/drive AXI signals internal to an FPGA at run time



51

Apply Your Knowledge

Q What is the purpose of the VIO core?

- ○ To monitor and drive internal FPGA signals in real time
- ○ To sample and store signals using on-chip block RAM
- ○ To provide advanced trigger equations for debugging
- ○ To connect external devices to the FPGA via AXI interface

52

Apply Your Knowledge

Q What debug cores are supported in the Vivado™ IP catalog? (Select all that apply)

- ☐ Integrated Logic Analyzer
- ☐ Virtual Input/Output
- ☐ Debug Switch
- ☐ AXI to JTAG Master
- ☐ IBERT

AMD
together we advance_

53

Summary

01 ILA debug core is used to monitor the design

02 VIO debug core inserts “virtual” pins into the design

03 Debug hub core is used to connect debug cores to the JTAG scan chain

04 Debug bridge provides connectivity to debug hub and debug cores

05 IBERT debugs, verifies, and optimizes high-speed serial communication channels

AMD
together we advance_

54

Netlist Insertion Debug Probing Flow

2023.2

AMD
together we advance_

55

Objectives

After completing this module, you will be able to:

OBJECTIVE 01

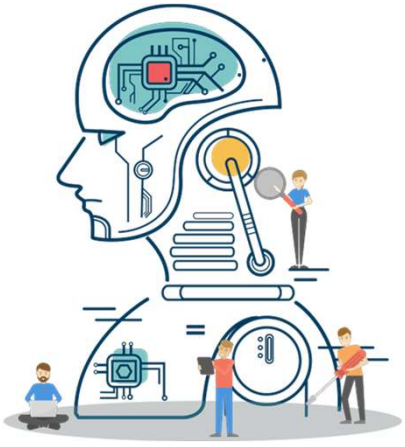
Describe the process of inserting the ILA core into a synthesized design netlist in the Vivado™ IDE

OBJECTIVE 02

Identify the strengths and weaknesses of the netlist insertion flow

OBJECTIVE 03

Use Tcl commands to insert debug cores and analyze the design



AMD
together we advance_

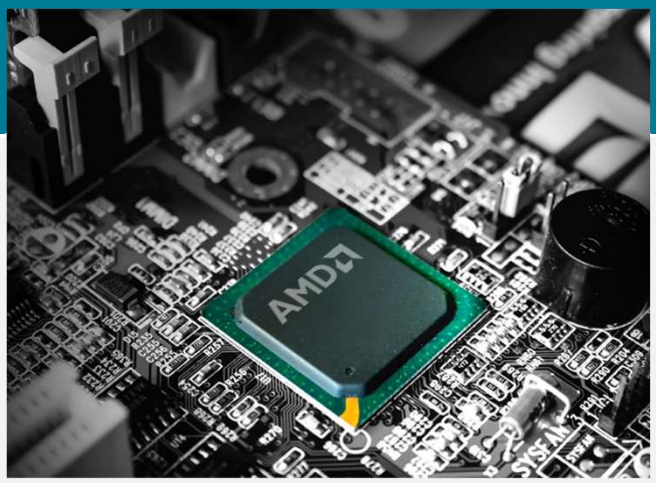
56

Introduction to the Netlist Insertion Flow

The Vivado™ IDE enables in-system debugging using the Vivado logic analyzer (VLA) through two flows:

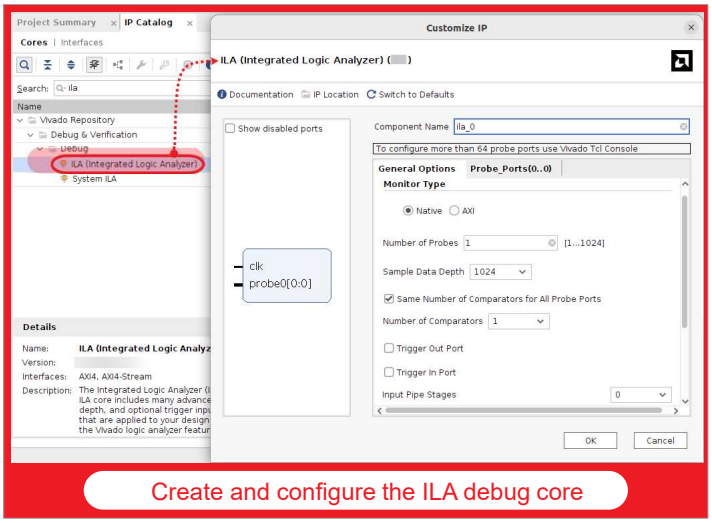
- HDL instantiation flow
- Netlist insertion flow

In the netlist insertion flow, the debug core is inserted into the synthesized netlist of a design



57

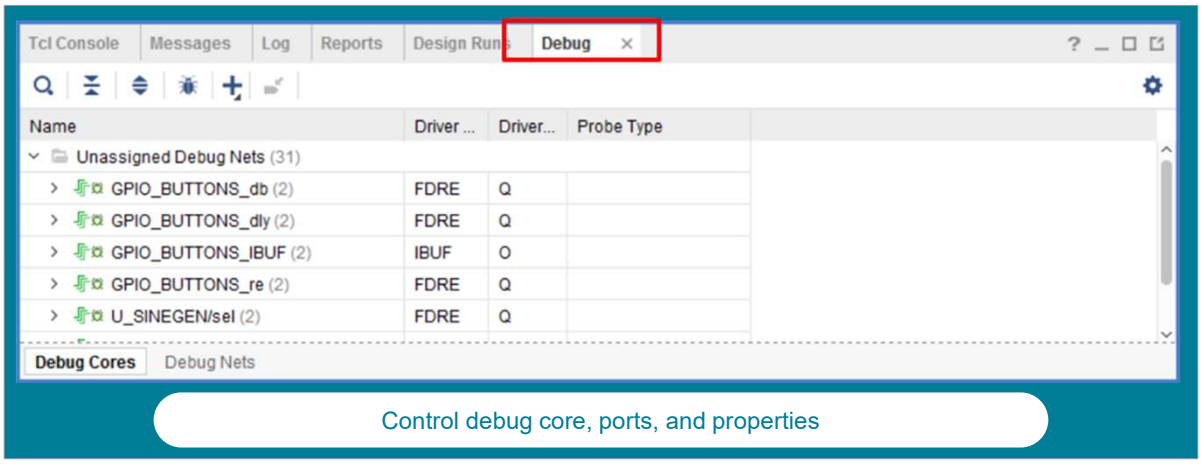
Introduction to the Netlist Insertion Flow



Create and configure the ILA debug core

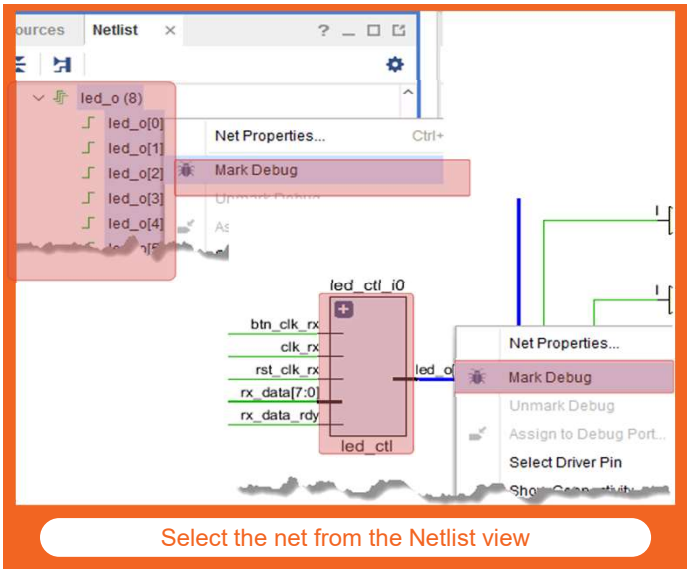
58

Introduction to the Netlist Insertion Flow



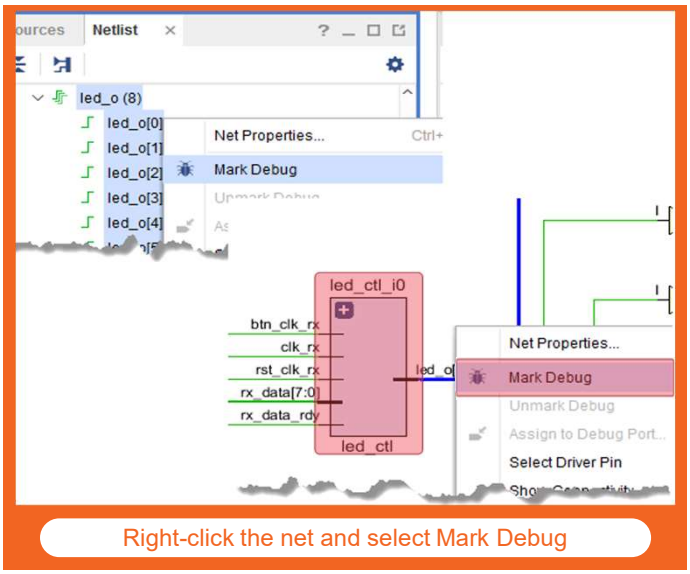
59

Introduction to the Netlist Insertion Flow



60

Introduction to the Netlist Insertion Flow



Right-click the net and select Mark Debug

AMD
together we advance_

61

Introduction to the Netlist Insertion Flow

Once the net is marked for debug:

Vivado™ Design Suite adds the net name to a temporary folder, which can be looked at by the Set Up Debug Wizard

Set Up Debug Wizard also enables the use of Unassigned Nets folder

Designer can use this temp folder to select the nets

Find capability allows users to search nets of interest in the dialog box

Netlist insertion flow is the most flexible flow as it allows probing at different design levels

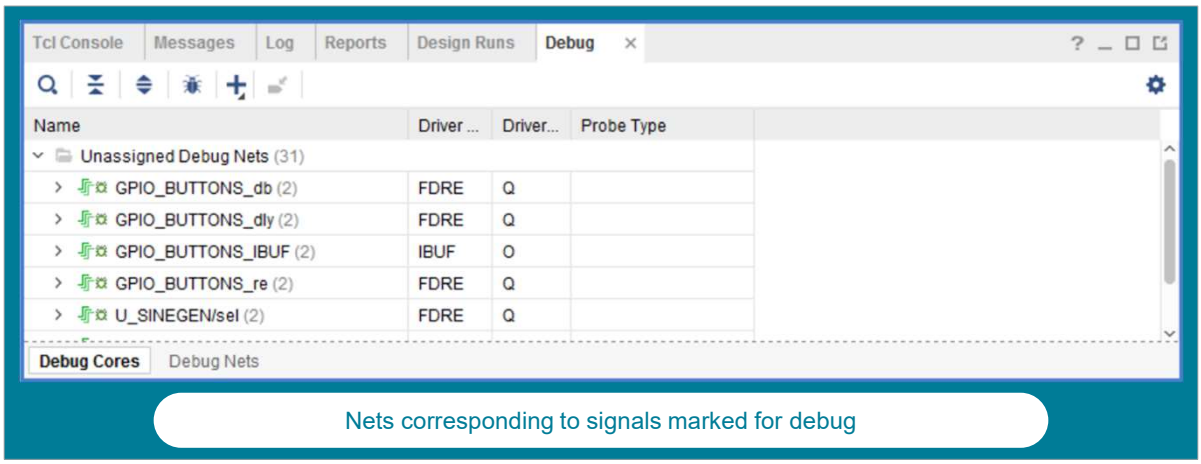
Only the ILA core can be added using this flow

AMD
together we advance_

62

Using the MARK_DEBUG Attribute

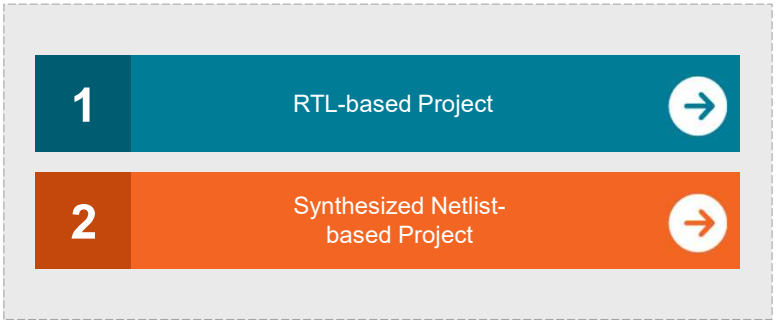
Identify signals for debugging at the HDL source level prior to synthesis using the MARK_DEBUG constraint



63

Using the MARK_DEBUG Attribute

Procedure for marking nets for debug depends on two types of projects



64

Using the MARK_DEBUG Attribute

RTL Netlist-based Project

Vivado™ synthesis feature to mark HDL signals for debug using MARK_DEBUG

VHDL

```
attribute MARK_DEBUG : string;
attribute MARK_DEBUG of char_fifo_dout: signal is "true";
```

Verilog

```
(* MARK_DEBUG = "true" *) wire [7:0] char_fifo_dout;
```

- Valid values for the MARK_DEBUG constraint are “TRUE” or “FALSE”
- Vivado synthesis feature does not support the “SOFT” value

65

Using the MARK_DEBUG Attribute

Synthesized Netlist-based Project

Use the MARK_DEBUG attribute without editing the HDL source files

Add the `MARK_DEBUG` attribute to the target XDC file:

```
set_property MARK_DEBUG true [get_nets sine*]
```

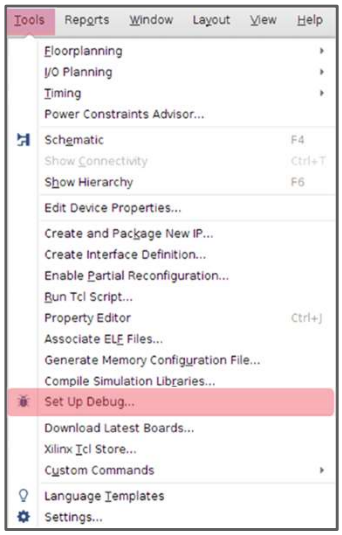
- Add MARK_DEBUG property on the nets using the synthesized Netlist view

66

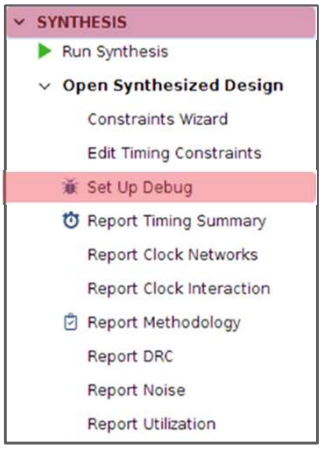
Set Up Debug Wizard to Insert Debug Cores

Once nets are marked for debug, they need to be assigned to debug cores

Set Up Debug Wizard helps in creating debug cores and assigning the debug nets to the inputs of debug cores



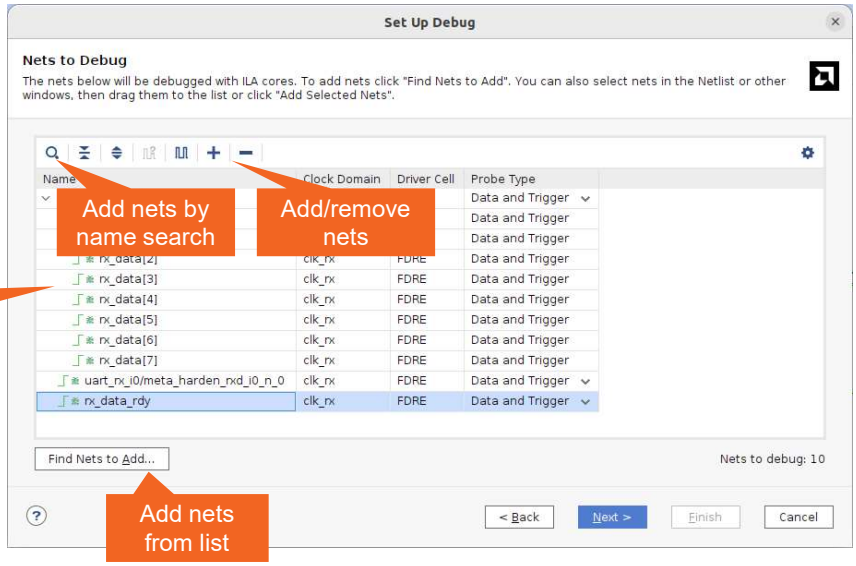
OR



AMD together we advance_

67

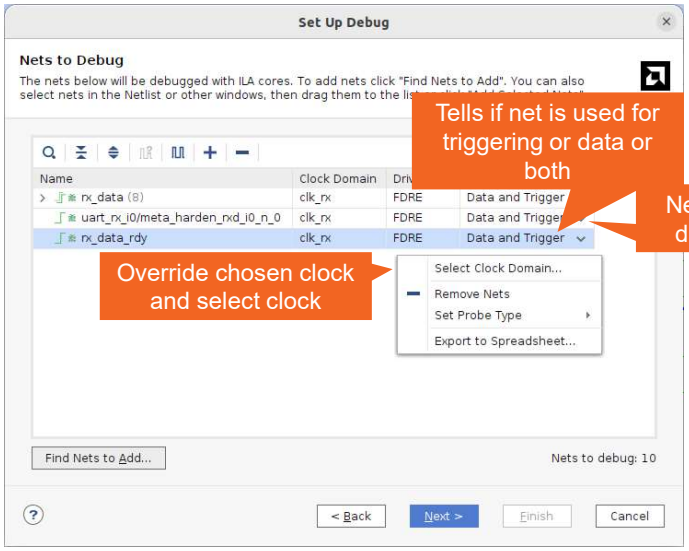
Set Up Debug Wizard to Insert Debug Cores



AMD together we advance_

68

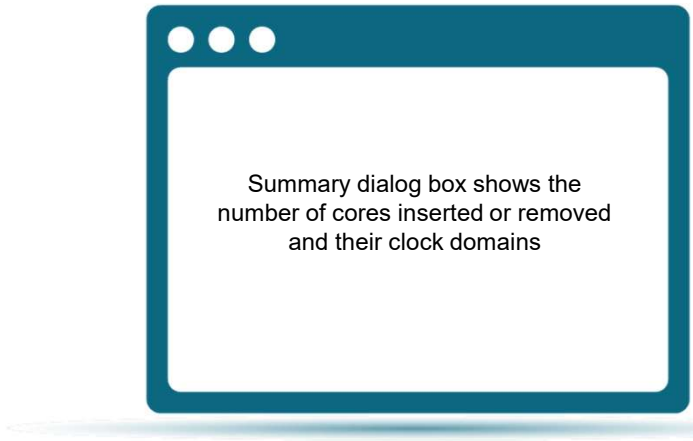
Set Up Debug Wizard to Insert Debug Cores



AMD together we advance_

69

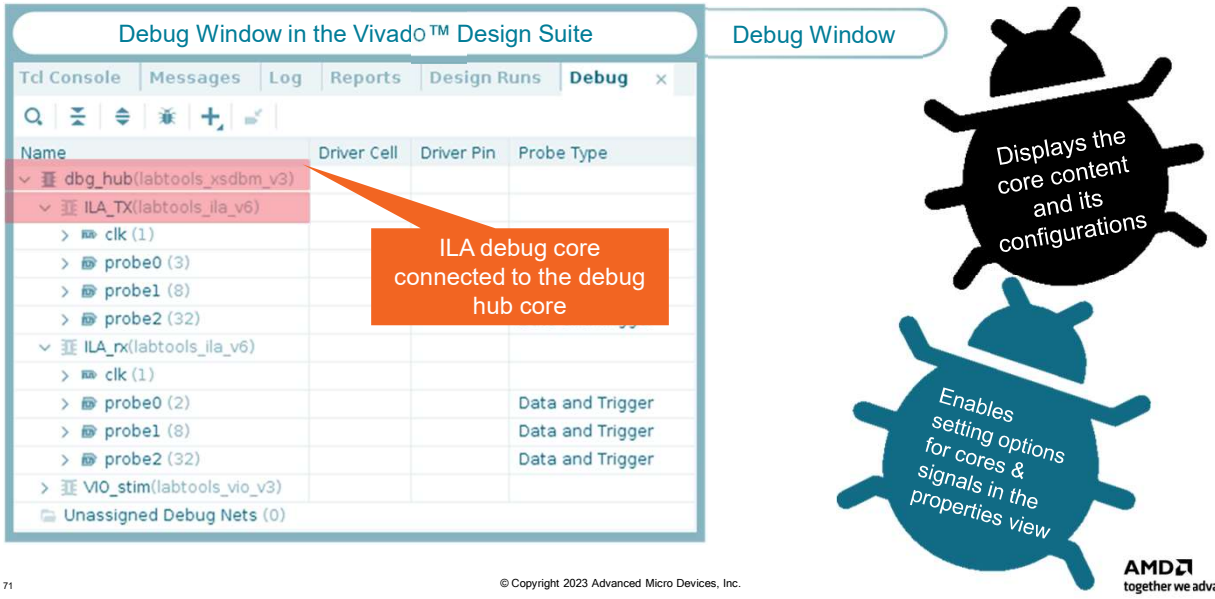
Set Up Debug Wizard to Insert Debug Cores



AMD together we advance_

70

Debug Core Customization in Debug Window

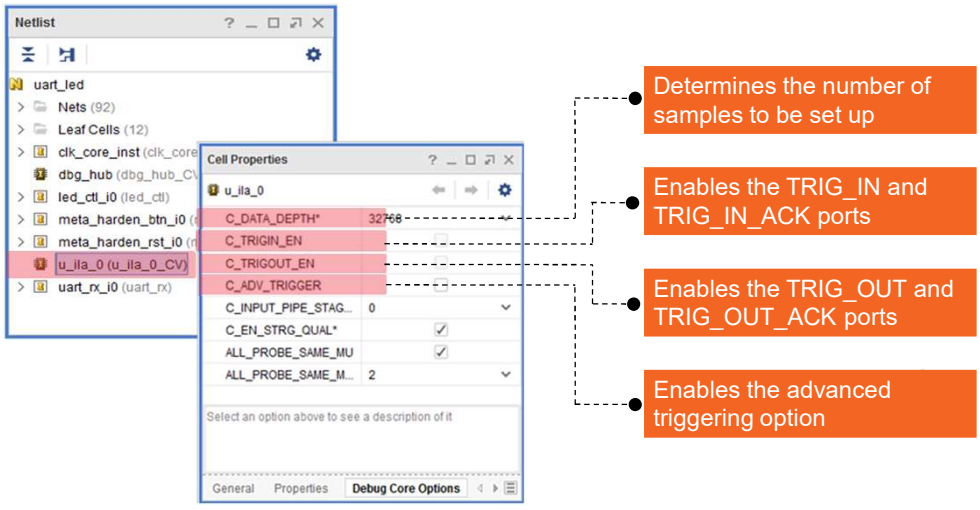


AMD together we advance_

71

Debug Core Customization in Debug Window

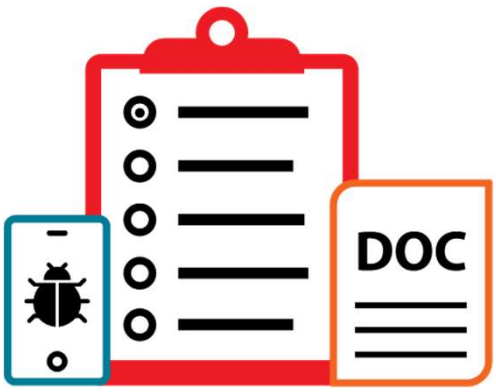
To see the Properties view, select debug core u_ila_0 > Debug Core Options in the Instance Properties window



AMD together we advance_

72

Tcl Commands to Insert Debug Cores



73

Tcl Commands to Insert Debug Cores

• create_debug_core u_ila_0 labtools_ila_v2	• Creates an ILA v2.1 core black box • Creates the CLK port and first probe PROBE0 port of the ILA v2.x core automatically
• create_debug_port u_ila_0 PROBE • set_property port_width 2 [get_debug_ports u_ila_0/PROBE1] • connect_debug_port u_ila_0/PROBE1 [get_nets [list {A[0]} {A[1]}]]	• Create more probe ports and set their width to 2 and connect them to the nets to debug
• set_property C_DATA_DEPTH 1024[get_debug_cores u_ila_0] • set_property port_width 1 [get_debug_ports u_ila_0/CLK] • set_property port_width 2 [get_debug_ports u_ila_0/PROBE0]	• Set the data depth of a port to 1024 and set the width of a clock to 1 and width of probe0 to 2

74

Tcl Commands to Insert Debug Cores

• connect_debug_port u_ila_0/CLK [get_nets [list clk]]	• Connects the clock port of the ILA to the desired clock net
• connect_debug_port u_ila_0/PROBE0 [get_nets [list A_or_B]]	• Connects the probe to the nets to be debugged

75

Implementing Debug Cores in the Vivado IDE

Debug core implementation is done automatically, but can be forced manually for floorplanning/timing analysis

Tcl command for the manual implementation of the debug core	implement_debug_core [get_debug_cores]

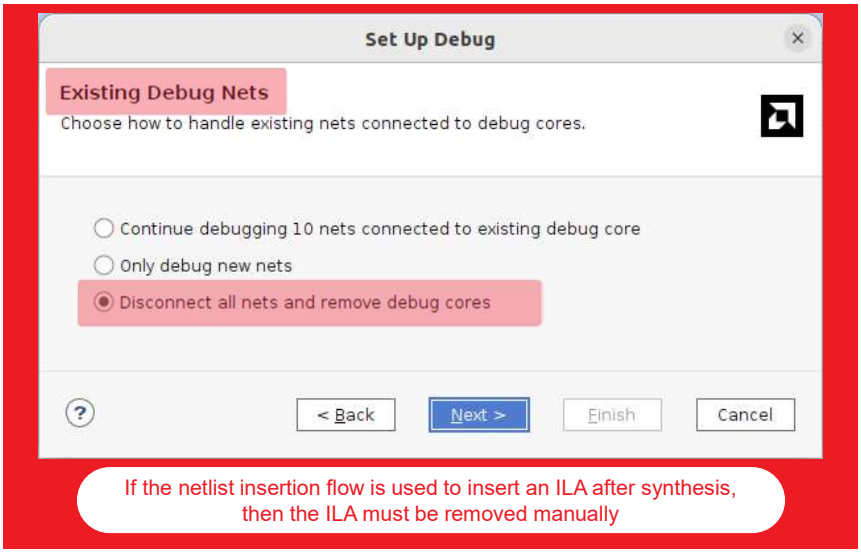
Points to be Considered While Implementing the Debug Core

Vivado™ IDE generates and synthesizes a black-box debug core	Netlists for cores are added to the project	Cores are implemented before the tool launches	Cores are used in all implementation runs	Core connectivity is preserved through netlist updates

After debug core implementation, debug core black boxes are resolved for accessing the generated instances

76

Removing Debug Logic after Debug



If the netlist insertion flow is used to insert an ILA after synthesis, then the ILA must be removed manually

AMD together we advance_

77

Configuring the Number of Comparators and Input Pipe Stages

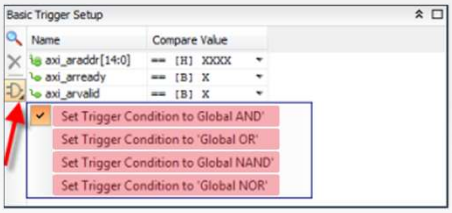
ILA used in the netlist insertion flow allows users to control the number of comparators & the input pipe stage

Number of Comparators

Input Pipe Stage

ILA probe trigger comparators or match units detect specific equality or inequality conditions on the probe inputs

Trigger condition is the result of a Boolean calculation of ILA probe trigger comparator results



AMD together we advance_

78

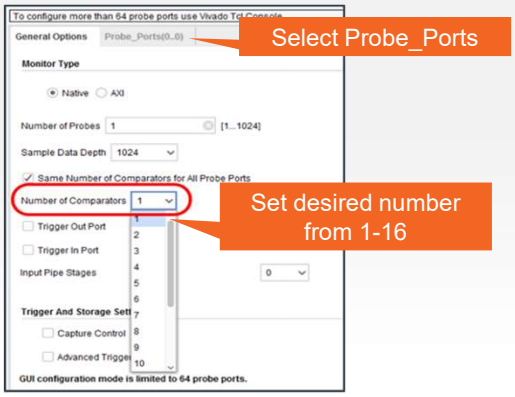
Configuring the Number of Comparators and Input Pipe Stages

ILA used in the netlist insertion flow allows users to control the number of comparators & the input pipe stage

Number of Comparators

Input Pipe Stage

ILA allows setting the number of comparators to be used per probe



AMD together we advance_

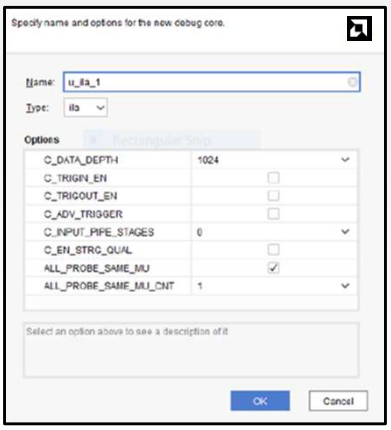
79

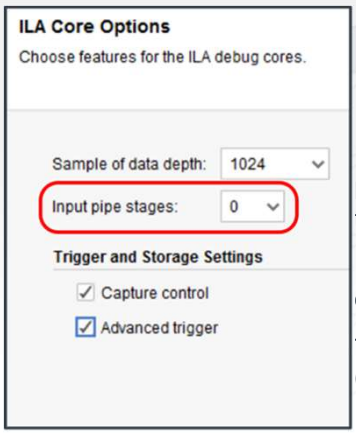
Configuring the Number of Comparators and Input Pipe Stages

ILA used in the netlist insertion flow allows users to control the number of comparators & the input pipe stage

Number of Comparators

Input Pipe Stage





Input Pipe Stage

Allows selection of the number of registers to be added for the probe

Enables extra levels of pipe stages on the PROBE inputs

Improves the timing performance of the design

Takes all valid values from 0 (default) to 6

AMD together we advance_

80

Apply Your Knowledge

Q The netlist insertion flow inserts a debug core into _____.

- An implemented netlist
- A synthesized netlist
- HDL code
- A bitstream file

AMD together we advance_

81

Apply Your Knowledge

Q What does the input pipe stage of the ILA core allow you to configure?

- The number of comparators used per probe
- The number of clock cycles to capture for each trigger event
- The number of registers added for each probe
- The number of clock domains used by the ILA core

AMD together we advance_

82

Summary

01 In the netlist insertion flow, the debug cores are added to the synthesized netlist

02 The Set Up Debug Wizard is used to create and configure a debug core

03 The Vivado™ IDE generates and synthesizes a black box debug core

AMD
together we advance_

GENERAL DISCLOSURE AND ATTRIBUTION STATEMENT

AMD
together we advance_

85